

普及组初赛模拟卷 18 解析

(C++语言 1.5 小时完成)

一、单项选择题 (共 20 题, 每题 2 分, 共计 40 分, 每题有且仅有一个正确答案。)

1. 已知小写字母 b 的 ASCII 码的十进制表示为 98, 那么大写字母 D 的 ASCII 码十六进制表示值是 (A)

- A. 44 B. 64 C. 68 D. 100

2. 下列有理数中, 在 10 进制下是有限小数而在 16 进制下是无限小数的是 (D)

- A. 7/8 B. 7/11 C. 7/16 D. 7/20

解析: 10 进制下的有限小数, 分母中可以有质因数 2 和 5, 2^k 进制下的有限小数, 分母里只能有质因数 2。

3. 假如某个局域网网关的 IPv4 地址为 192.68.3.254/24, 那么这个局域网最多能够接入 (A) 台其他设备 (一般 0 和 255 不用做设备的 IP 地址)

- A. 253 B. 254
C. 255 D. 256

解析: 能用的 IPv4 地址范围是 192.68.3.1 到 192.68.3.253, 共 253 台设备。

4. 在 C++ 中定义如下数组 `unsigned int a[]={1,2,3,4,5}`, 存储容量为 (C)

- A. 5bit B. 40bit C. 20B D. 40B

5. 下列各种说法中, 错误的是 (B)

- A. 计算机软件可以分为系统软件和应用软件
B. 系统软件的运行依赖于应用软件的支持
C. 操作系统属于系统软件, 是计算机中不可或缺的部分
D. 常见的应用软件有 Photoshop、Word、微信等

解析: B 选项应该反过来, 应用软件的运行依赖于系统软件的支持。

6. 下列计算机网络相关协议与其工作的层级相对应的是正确的是 (C)。

- A. IP 协议: 传输层 B. TCP 协议: 网络层
C. HTTP 协议: 应用层 D. FTP 协议: 数据链路层

解析: IP 协议工作在网络层, TCP 协议工作在传输层, FTP 协议工作在应用层。

7. C++ 中若变量 a 和 b 的初始值为 1 和 2, 则执行语句 `a+=b++`; 后 a 和 b 的值分别为 (B)

- A. 2 3 B. 3 3 C. 4 2 D. 4 3

解析: `b++` 运算是先返回 b 的值再加 1; `++b` 运算是先加 1 再返回 b 的值。

8. 若整型变量 x 的值为 26, 下列选项中, 则表达式 `x&(x-1)` 的值为 (C)

- A. 18 B. 21 C. 24 D. 27

解析: `&` 为位运算, 当且仅当两边的二进制位同时为 1 时取 1, $26 = (11010)_2$, $25 = (11001)_2$, 因此 `x&(x-1) = (11000)_2 = 24`。

9. 前缀表达式 `*a-bcd` 中, $a=4, b=3, c=2, d=1$, 则该前缀表达式的值是: (C)

- A. -3 B. 1 C. 5 D. 11

解析：前缀表达式其实就是表达式树的前序遍历，把表达式树画出来即可（所有变量为叶节点，运算符为非叶节点）。该表达式为 $a+(b-c)*d=5$ 。

10. 将洗完的餐盘从下往上叠起来，用的时候从上往下取餐盘，其原理和下列那种数据结构类似：（ B ）

- A. 队列 B. 栈 C. 树 D. 链表

11. 对 n 个元素进行选择排序，则排序过程中的交换次数最多多少次：（ B ）

- A. $n/2$ B. $n-1$ C. n D. $n(n-1)/2$

解析：选择排序每趟排序只有最大的数或者最小的数才会发生交换，排序总共进行 $n-1$ 轮，最多交换 $n-1$ 次。

12. 对序列【3,2,6,5,1,8,4】进行冒泡排序(升降序及左右方向未知)，则在经过 2 轮排序之后，序列的中间结果不可能为（ A ）

- A. 1,2,3,6,4,5,8 B. 2,3,1,5,4,6,8 C. 6,5,3,8,4,2,1 D. 8,6,3,2,5,4,1

解析：题目中没说明到底是升序还是降序，从左向右排序还是从右向左排序，那就把这 4 种情况全部枚举出来即可。

从左向右的升序排序：

开始： 3 2 6 5 1 8 4

第一轮：2 3 5 1 6 4 8

第二轮：2 3 1 5 4 6 8

从左向右的降序排序：

开始： 3 2 6 5 1 8 4

第一轮：3 6 5 2 8 4 1

第二轮：6 5 3 8 4 2 1

从右向左的升序排序：

开始： 3 2 6 5 1 8 4

第一轮：1 3 2 6 5 4 8

第二轮：1 2 3 4 6 5 8

从右向左的降序排序：

开始： 3 2 6 5 1 8 4

第一轮：8 3 2 6 5 1 4

第二轮：8 6 3 2 5 4 1

因此答案选 A。

13. 归并排序最优情况下的时间复杂度为：（ C ）

- A. $O(\log_2^n)$ B. $O(n)$ C. $O(n\log_2^n)$ D. $O(n^2)$

解析：归并排序不管是最优情况还是最坏情况，合并两个有序数组的时间复杂度都是一样的，等于这两个数组的长度之和为 $O(n)$ ，又因为分治的过程递归了 $\log_2 n$ 层，因此整体的时间复杂度为 $O(n\log_2^n)$ 。

14. 某算法的计算时间函数 $T(n)$ 满足 $T(n)=2T(n/2)+1$ ，则该算法的时间复杂度为：（ B ）

- A. $O(\log_2^n)$ B. $O(n)$ C. $O(n\log_2^n)$ D. $O(n^2)$

解析：这里两边同时加上 1 即可。 $T(n)+1=2(n/2)+2=2(T(n/2)+1)$ 。

因此， $T(n)+1=2^{\log_2 n}=n$ 。故 $T(n)=n-1=O(n)$ 。

15. 若按照 a,b,c,d,e,f 的顺序依次出栈，则下列选项中 (D) 不是可能的入栈顺序

A. a,b,c,d,e,f B. f,e,d,c,b,a C. d,c,a,b,f,e D. b,a,d,e,c,f

16. 若一颗二叉树的中序遍历为 d,b,f,e,a,c,g, 后序遍历为 d,f,e,b,g,c,a, 则该二叉树的前序遍历为 (D)

A. a,b,c,d,e,f,g B. a,b,c,d,e,g,f C. a,b,d,f,e,c,g D. a,b,d,e,f,c,g

17. 已知一棵完全二叉树一共有 2022 个节点，则其叶节点数量为 (C)

A. 998 B. 999 C. 1011 D. 1012

解析：完全二叉树的叶节点个数等于总节点数除以 2 向上取整。

18. 12 以内的正整数 (包含 12) 互质的数共有 (B) 对。(注：(a,b)和(b,a)算同一对)

A. 45 B. 46 C. 47 D. 48

解析：我们不妨设 a 小于 b 好了，由于互质的数比不互质的数要多，我们只考虑不互质的数。

当 a=2 时，b=4,6,8,10,12。

当 a=3 时，b=6,9,12。

当 a=4 时，b=6,8,10,12。

当 a=5 时，b=10。

当 a=6 时，b=8,9,10,12。

当 a=8 时，b=10,12。

当 a=9 时，b=12。

当 a=10 时，b=12。

另外，有一种特殊情况是 1 与 1 互质，因此答案等于 $12+11*10/2-21=46$ 。

19. 黑白两种棋子共 3000 枚，分成 1000 堆，每堆 3 枚。其中只有 1 枚白子的共 270 堆，至少有 2 枚黑子的共 420 堆，有 3 枚白子的与 3 枚黑子的堆数相同。则白棋共 (C) 枚。

A. 1360 B. 1420 C. 1580 D. 1640

解析：我们设 3 白有 a 堆，2 白 1 黑有 b 堆，1 白 2 黑有 c 堆，3 黑有 d 堆，则：

$$a+b+c+d=1000$$

$$c=270$$

$$c+d=420$$

$$a=d$$

因此：a=150,b=430,c=270,d=150。白棋有 $3a+2b+c=1580$ 枚。

20. 下列哪位计算机科学家为现代信息论的建立奠定了基础： (B)

A. 冯·诺依曼 B. 克劳德·香农 C. 戈登·摩尔 D. 阿兰·图灵

二、 阅读程序 (程序输入不超过数组或字符串定义的范围：判断题正确填 T，错误填 F；除特殊说明外，判断题 2 分，选择题 3 分，共计 30 分)

1.

```
1  #include <bits/stdc++.h>
2  using namespace std;
3  int main(){
```

```

4     double a,b,c;
5     cin>>a>>b>>c; //输入保证 a,b,c 均为正数
6     double s = 0;
7     if (a*a+b*b==c*c) s = a*b/2.0;
8     if (a*a+c*c==b*b) s = a*c/2.0;
9     if (b*b+c*c==a*a) s = b*c/2.0;
10    if (s>0) cout<<s;
11    else {
12        if (a+b>c) s++;
13        if (a+c>b) s++;
14        if (b+c>a) s++;
15        cout<<s;
16    }
17    return 0;
18 }

```

判断题

- 1) 若该程序的输出结果大于 3, 则以 a,b,c 为三边边长能构成直角三角形。(T)
- 2) 若该程序的输出结果小于 3, 则以 a,b,c 为三边边长不能构成三角形。(F)

选择题

- 3) 若输入的值依次为 "13 5 12", 则程序的输出为: (A)
 A. 30 B. 32.5 C. 60 D. 65
- 4) 若该程序的输入数据均为不大于 30 的正整数, 则输出 s 的最大值为: (C)
 A. 120 B. 180 C. 216 D. 240

解析: 1) 若第 10 行中 s 大于零, 则说明以 a,b,c 能构成, 否则进入到 else 条件, 而如果进入了 else 条件则 s 的值是不可能超过 3 的, 因此输出结果大于 3, 则以 a,b,c 为三边能构成直角三角形。

2) 如果 a,b,c 构成了直角三角形, 并且面积小于 3 的话, 输出的结果 s 也会比 3 小。

3) 由于 $5*5+12*12=13*13$, 因此构成直角三角形, 面积 $s=5*12/2=30$ 。

4) 这道题直接枚举的话情况太多了, 可以反向思考。s 取最大值时显然构成直角三角形, 不妨设 $a < b < c$, 那么有 $ab=2s$, 由于 a 和 b 都是 30 以内的正整数, 我们把 2s 分解质因数就可以了。

D 选项中 $2s=480=16*30=20*24$, 代入验证都不可行。

C 选项中 $2s=432=16*27=18*24$, 代入验证当 $a=18, b=24$ 时, $c=30$, 构成直角三角形。

2.

```

1  #include <bits/stdc++.h>
2  using namespace std;
3  int a[1001][1001];
4  int main()
5  {
6      int n,m,p;
7      cin>>n>>m>>p;
8      for(int i = 0; i <= n; i++) {
9          a[i][0] = 1;
10         for(int j = 1; j <= i; j++) {
11             a[i][j]=a[i-1][j-1]+a[i-1][j];
12             if(a[i][j]>=p)

```

```

13         a[i][j]%=p;
14     }
15 }
16 cout<<a[n][m];
17 return 0;
18 }

```

判断题

- 1) 该程序的时间复杂度为 $O(nm)$ 。(F)
- 2) 第 13 行的代码改为 “a[i][j]-=p;” 不影响程序的正确性。(T)

选择题

- 3) 输入 1:6 4 30 (B)

A. 10	B. 15
C. 20	D. 0
- 4) 输入 2: 500 300 13 (D)

A. 1	B. 2	C. 6	D. 12
------	------	------	-------

解析：这道题目考察了利用杨辉三角形求组合数的算法。

1) 算法两重循环第一重范围 0 到 n，第二重范围 1 到 i，所以时间复杂度是 $O(n^2)$ ，和 m 无关。

2) 在计算过程中，若 $a[i-1][j-1] < p$ 且 $a[i-1][j] < p$ ，那么两者加起来 $a[i][j] = a[i-1][j-1] + a[i-1][j] < 2p$ ，因此直接减 p 的效果和求余数是一样的。

3) 组合数 $C(6,4) = 6! / (4! * 2!) = 15$ ，也可以用杨辉三角的方法计算。

4) 组合数 $C(500,300) \bmod 13$ ，我们发现 n 和 m 很大，而模数 13 很小，并且是个质数。我们可以利用卢卡斯(Lucas)定理进行计算，首先由于对称性 $C(500,300) = C(500,200)$ 。在 13 进制下：

$$200 = (1 \ 2 \ 5)_{13}$$

$$500 = (2 \ C \ 6)_{13}$$

$$\text{因此, } C(500,200) = C(2,1) * C(12,2) * C(6,5) = 2 * 66 * 6 = 12 \pmod{13}$$

3.

```

1  #include <bits/stdc++.h>
2  using namespace std;
3  int sum;
4  int a[2001];
5  int select(int l, int r, int k) {
6      sum++;
7      int m = a[(l+r)/2], t;
8      int i = l, j = r;
9      while(i<=j) {
10         while (a[i]>m) i++;
11         while (a[j]<m) j--;
12         if(i<=j) {
13             t = a[i]; a[i] = a[j]; a[j] = t;
14             i++; j--;
15         }
16     }
17     if (k<=j) return select(l,j,k);
18     if (k>=i) return select(i,r,k);

```

```

19     return a[k];
20 }
21 int main()
22 {
23     int n,k;
24     cin>>n>>k;
25     sum = 0;
26     for(int i=1;i<=n;i++) cin>>a[i];
27     cout<<select(1,n,k)<<' ';
28     cout<<sum<<endl;
29     return 0;
30 }

```

判断题

1) 将程序第 12 行的条件 “ $i \leq j$ ” 直接改为 true，不影响程序的运行结果。(F)

选择题

2) 该算法的平均时间复杂度为：(B) (2 分)

- A. $O(\log_2 n)$ B. $O(n)$ C. $O(n \log_2 n)$ D. $O(n^2)$

3) 输入 1: 10 3 6 2 1 8 7 9 4 5 10 3

输出 1: (D)

- A. 3 3 B. 3 4 C. 8 3 D. 8 4

4) 输入 2: 1000 13 1 2 3 4 ... 998 999 1000

输出 2: (C)

- A. 13 10 B. 13 11 C. 988 9 D. 988 10

解析：这道题目考察了利用分治思想（类似于快速排序）求第 k 大数的算法。

1) 如果把 12 行这个条件去掉，那么 i 有可能是比 j 要大的，因为如果前面的循环中 i 和 j 都恰好越过了中位数，那么 i 在 j 的右边，此时交换 $a[i]$ 和 $a[j]$ 反而把小数换到左边去了，导致程序的结果可能出现错误。

2) 平均情况下，每次循环完区间被分为原来的一半，而且递归进入到下一层循环情况是随机的。因此平均情况下的时间复杂度为 $n + n/2 + n/4 + \dots + 1 \leq 2n - 1 = O(n)$ 。

3) 数据范围不大，直接模拟一下就行了。

开始：6 2 1 8 7 9 4 5 10 3

第一轮 $l=1, r=10$ ，结束后：10 9 7 8 1 2 4 5 6 3 此时 $i=5, j=4$

第二轮 $l=1, r=4$ ，结束后：10 9 7 8 此时 $i=3, j=1$

第三轮 $l=3, r=4$ ，结束后：8 7 此时 $i=4, j=3$

第四轮 $l=3, r=3$ ，找到答案 8 结束，输出 8 4。

4) 虽然数据范围很大，但是其实第一轮结束后就全部排好序了。因此只需要考虑 l, r, i, j 的值即可。

第一轮： $l=1, r=1000, i=501, j=500$

第二轮： $l=1, r=500, i=251, j=249$

第三轮： $l=1, r=249, i=126, j=124$

第四轮： $l=1, r=124, i=63, j=61$

第五轮： $l=1, r=61, i=32, j=30$

第六轮： $l=1, r=30, i=16, j=14$

第七轮: $l=1, r=14, i=8, j=6$

第八轮: $l=8, r=14, i=12, j=10$

第九轮: $l=12, r=14$, 此时第 k 位置恰好是中位数, 直接输出结果 988 9。

三、完善程序 (单选题, 每空 3 分, 共 30 分)

1.

(后缀表达式求值) 输入一个后缀表达式, 计算表达式的值, 计算结果保留三位小数。例如, 后缀表达式 “5 8 2 - 2 * 3 / +”, 首先计算 $8-2$ 等于 6, 然后计算 $6*2$ 等于 12, 然后计算 $12/3$ 等于 4, 最后计算 $5+4=9$ 。

【输入】

一行一个字符串, 表示后缀表达式, 数据保证所有的数字都是一位整数, 并且数字和运算符号之间都用一个空格隔开。(运算符包括加减乘除 4 种)

【输出】

一行一个实数, 保留 3 位小数, 表示运算结果。

样例输入:

5 8 2 - 2 * 3 / +

样例输出:

9.000

```
#include <bits/stdc++.h>
using namespace std;
double st[100];
double calc(double a, double b, char op) {
    if (op=='+') return a+b;
    if (op=='-') return a-b;
    if (op=='*') return a*b;
    return ①_____ ;
}
int main()
{
    int n,top = -1;
    double a,b,c;
    char s[30];
    ②_____ ;
    n = strlen(s);
    for (int i = 0; i < n; i+=2)
        if (s[i]>='0' && s[i]<='9') {
            top++;
            st[top] = s[i]-'0';
        } else {
            a = st[top]; top--;
            b = st[top]; top--;
            c = ③_____ ;
            ④_____ ;
        }
    printf(⑤_____);
    return 0;
}
```

}

1) ①处应填: (A)

A. a/b

B. a%b

C. b/a

D. b%a

2) ②处应填: (C)

A. cin>>s

B. scanf("%s",s)

C. gets(s)

D. s=gets()

3) ③处应填: (D)

A. calc(a,b,i)

B. calc(a,b,s[i])

C. calc(b,a,i)

D. calc(b,a,s[i])

4) ④处应填: (B)

A. top--; st[top] = c;

B. top++; st[top] = c;

C. st[top] = c; top++;

D. st[top] = c; top--;

5) ⑤处应填: (B)

A. "%.3lf",st[top]

B. "%.3lf",st[top]

C. "%.3lf",st[top+1]

D. "%.3lf",st[top-1]

解析: 这道题目考察了利用栈的数据结构求解后缀表达式。

1) 送分题, 除了加减乘三种运算之外就是除法运算, 因此答案为 a/b。

2) 输入的表达式中数字和运算符是用空格隔开的, 直接用 cin 或者%s 的格式是不能输入空格的, 因此需要用 gets()函数来一整行数据。

3) 首先按照入栈的顺序 a 应该在 b 的后面, 因此两个运算的数先后顺序为先 b 后 a, 另外运算符 op 的类型为 char, 因此第三个参数为 s[i]。

4) 考查了入栈的操作, 由于 st[top]已经有值, 所以要先 top++, 在进行赋值。

5) %.3lf 为格式化输出, 表示保留三位小数, 而 3 写在小数点左边则表示场宽。

2.

(链表去重) 输入一个长度为 n 的不下降序列, 将其保存到单向链表之中, 并去掉其中的重复元素。例如, 输入序列 "1,2,2,3,3", 首先构建链表 "1->2->2->3->3", 然后从头到尾遍历链表, 删除重复的 2 和 3, 最后输出结果 1,2,3。

【输入】

第一行为 n, 表示序列的长度

第二行为 n 个空格隔开的正整数, 为序列元素, 保证输入的序列是不下降的。

【输出】

一行若干个正整数, 表示去重后的序列。

样例输入:

5

1 2 2 3 3

样例输出:

1 2 3

```
#include <bits/stdc++.h>
```

```
using namespace std;
```

```
int a[100][2]; //用数组 a 模拟链表
```

```
//a[i][0]表示元素值, a[i][1]表示 next 指针
```



```

int head; //head 为链表头指针
int tot; //tot 为链表元素插入的位置
void add(int t) { //在链表尾插入元素 x
    if (head==-1){
        a[tot][0] = t; a[tot][1] = ①_____；
        head = tot; tot++;
    } else {
        int x = head;
        while (a[x][1]!=-1) x = a[x][1];
        a[tot][0] = t; a[tot][1] = -1;
        ②_____； tot++;
    }
}
void del() { //去掉重复元素
    if (head==-1) return;
    int x = head, nextx = a[x][1];
    while (a[x][1]!=-1) {
        if (a[x][0]==a[nextx][0])
            ③_____；
        else x = a[x][1];
        ④_____；
    }
}
int main()
{
    int n,t;
    cin >> n;
    head = -1; tot = 0;
    for(int i = 0; i < n; i++) {
        cin >> t;
        add(t);
    }
    del();
    int x = head;
    while (x!=-1) {
        cout<<a[x][0]<<' ';
        ⑤_____；
    }
    return 0;
}

```

1) ①处应填: (B)

A. 0 B. -1 C. head D. tot

2) ②处应填: (D)

A. a[tot][0]=x B. a[tot][1]=x
C. a[x][0]=tot D. a[x][1]=tot

3) ③处应填: (C)

A. a[x][0] = a[nextx][0] B. a[nextx][0] = a[x][0]
C. a[x][1] = a[nextx][1] D. a[nextx][1] = a[x][1]

4) ④处应填: (D)

A. $x = a[x][1]$

B. $x = a[nextx][1]$

C. $nextx = a[x][1]$

D. $nextx = a[nextx][1]$

5) ⑤处应填: (B)

A. $x = a[x][0]$

B. $x = a[x][1]$

C. $x++$

D. $x += a[x][1]$

解析: 这道题目考察了单向链表的插入和删除操作。

1) 向空链表中插入元素, 这个元素没有后继节点, 因此指针 $a[tot][1]$ 的值赋值为 -1。

2) 向链表的结尾插入新元素, 原来的结尾 x 的指针 $a[x][1]$ 应该指向 tot 。

3) 考察了链表删除, 删除链表中的元素就是将它前驱的指针指向它的后继即可。

4) $nextx$ 这个节点已经访问过了, 接下来需要访问它的后继 $a[nextx][1]$ 。

5) 考察了链表的变量, $x=a[x][1]$ 即访问指针。